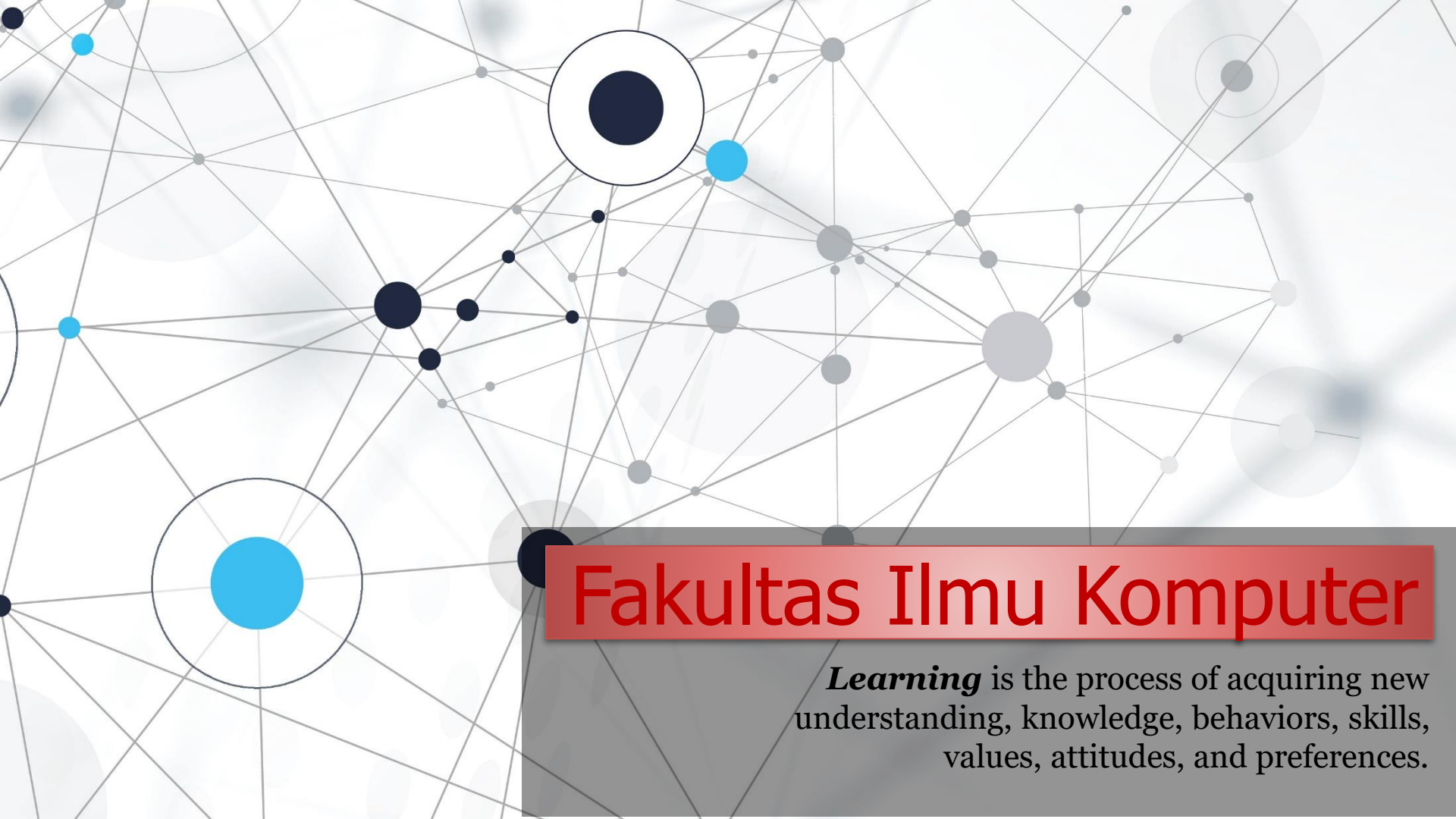


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

# Fakultas Ilmu Komputer

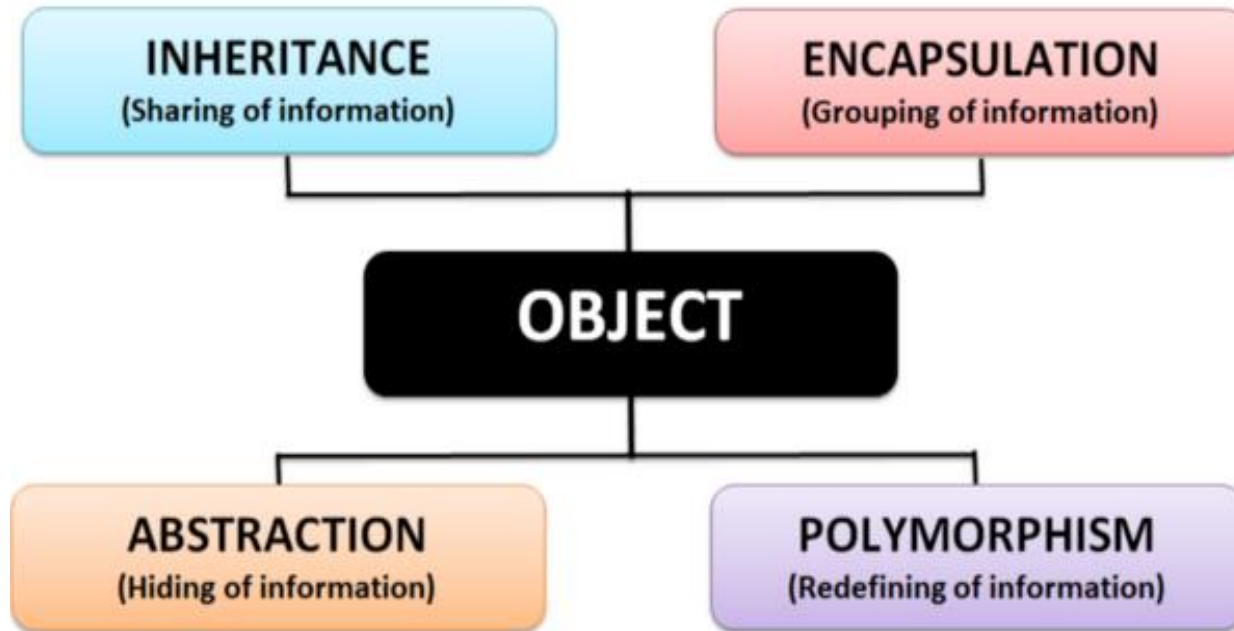
***Learning*** is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

## Python Encapsulation



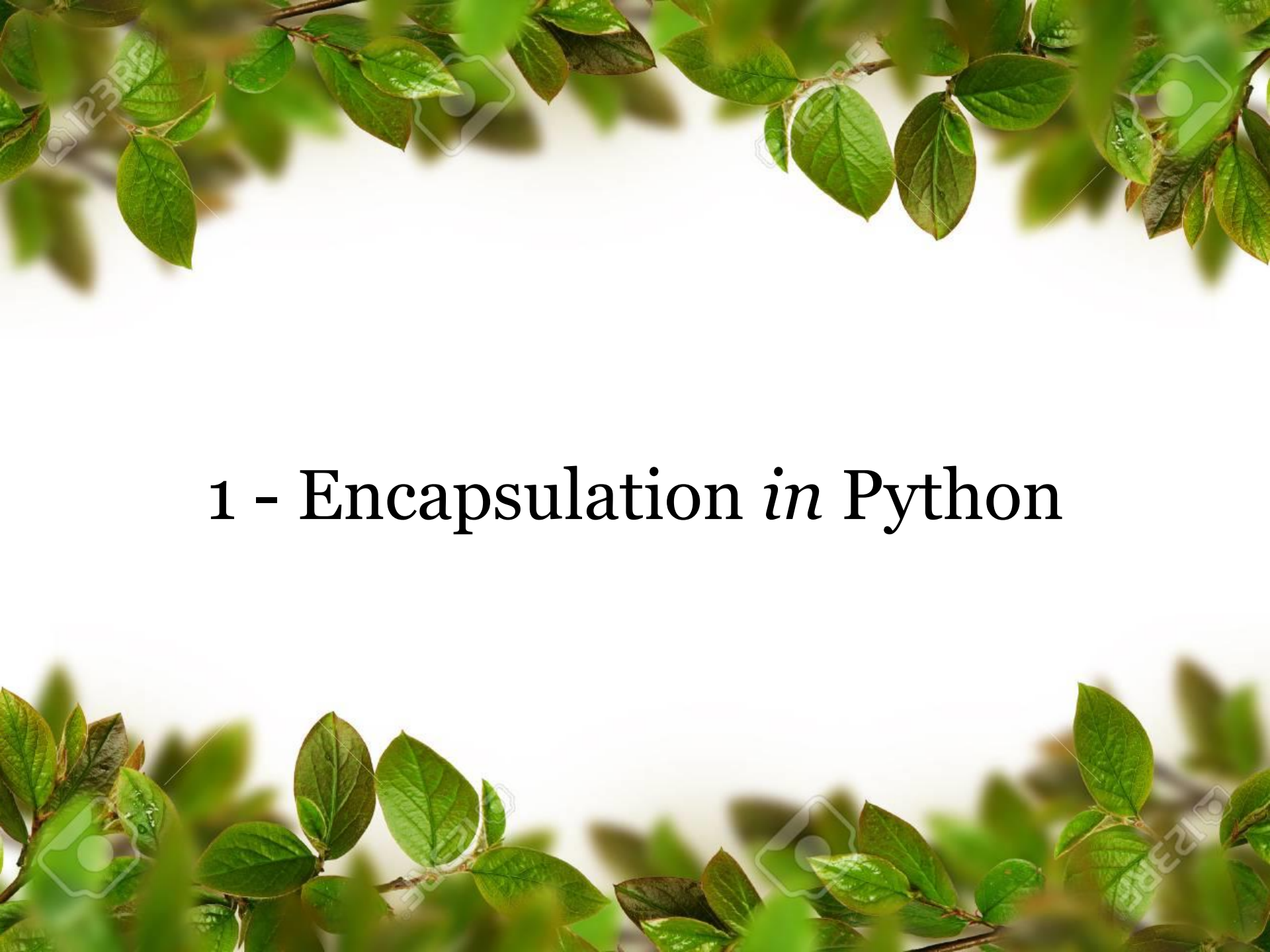
**Welcome to *the* Object Oriented  
Programming *course***

# Understanding *the* Concept of OOP



- ✓ Object Oriented Programming (OOP) atau pemrograman berorientasi objek adalah salah satu pendekatan paling efektif dalam menulis kode suatu program/software.
- ✓ Pada OOP, kita menulis class yang merepresentasikan suatu situasi seperti pada keadaan sebenarnya dan kita dapat membuat suatu objek berdasarkan class tersebut.

The four pillars of object-oriented programming in Python are: Abstraction; Encapsulation; Inheritance; Polymorphism.



# 1 - Encapsulation *in* Python

**Do you know, what is it?**



# Do you know, what is it?



# Do you know, what is it?



Itu adalah  
obat untuk  
hati yang  
tersakiti.



# What is Encapsulation?

**Encapsulation** is one of the fundamental concepts in object-oriented programming (OOP).

- 1) The **process of wrapping up variables and methods into a single entity** is known as Encapsulation. It is one of the underlying concepts in object-oriented programming (OOP). It acts as a **protective shield that PUTS RESTRICTIONS ON ACCESSING VARIABLES AND METHODS DIRECTLY**, and can prevent accidental or unauthorized modification of data.
- 2) It describes the **idea of wrapping data and the methods that work on data within one unit**. This **puts restrictions on accessing variables and methods** directly and can **prevent the accidental modification of data**. To prevent accidental change, an object's variable can only be changed by an object's method. Those types of variables are known as **private variables**.
- 3) **Encapsulation is used to restrict access to methods and variables**. In encapsulation, code and data are wrapped together within a single unit from **being modified by accident**.
- 4) Encapsulation is the **packing of data and functions that work on that data within a single object**.
- 5) **Encapsulation isn't a security measure in that it prevents people from messing (mengotak-atik) with our code**. It's a security measure in the sense that people can't go directly in and change variables without going through our proper channels.
- 6) Encapsulation is to a programming approach that revolves around data and functions contained, or encapsulated, within a set of operating instructions.

# What is Encapsulation?

- ❖ Enkapsulasi (Encapsulation) adalah **sebuah teknik dalam OOP yang mengizinkan kita untuk menyembunyikan detil dari sebuah atribut** dalam sebuah class.
- ❖ Encapsulation adalah salah satu konsep fundamental pada pemrograman berorientasi objek. Encapsulation menggambarkan suatu ide untuk membungkus data dan metode yang bekerja pada suatu data di dalam satu unit.
- ❖ Hal ini memberikan batasan dalam mengakses suatu variabel dan metode secara langsung. Sehingga, dapat mencegah modifikasi data yang tidak disengaja.

Ingat ini sangat penting, karena dapat **mencegah perubahan yang tidak disengaja**. Bukan tidak dapat di ubah, namun, variabel objek tersebut hanya dapat di ubah dengan metode objek.



**Menurut pendapat  
anda, apakah ada sistem  
yang benar-benar  
AMAN !!!**

Rumah tanpa pintu !!!



# Programming Technique *to* Restrict Access

## Tidak Ada Sistem Yang Aman !!!

- ❖ Encapsulation prevents data modification accidentally by limiting access to variables and methods.
- ❖ We can hide the internal state of the object from the outside. This is known as information hiding.



Setiap atribut dan fungsi yang telah kita definisikan belum menggunakan konsep enkapsulasi, yang mengartikan bahwa setiap atribut dan fungsi dapat diakses di luar class.

All member variables of the class are by default public.

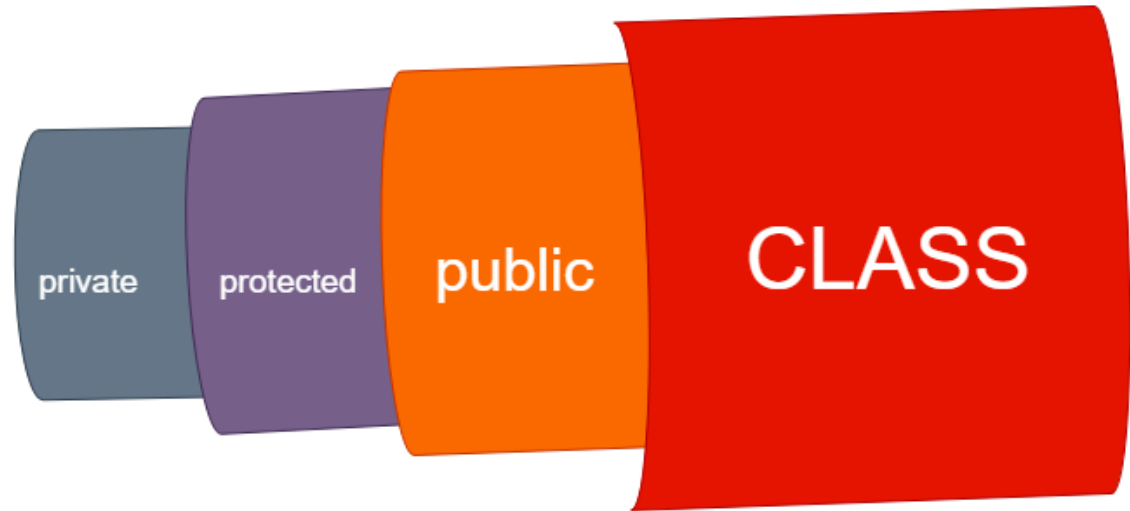
A decorative border of green leaves and branches at the top of the slide.

## 2 - Data Hiding using public, protected, & private members

A decorative border of green leaves and branches at the bottom of the slide.

# Access Modifiers *in* Python

Encapsulation can be achieved by declaring the data members and methods of a class either as **private or protected**. But In Python, **we don't have direct access modifiers/keywords like public, private, and protected**. We can achieve this by using single underscore and double underscores.



- ❖ **Access modifiers give limited access to the variables and methods** of a class.
  - ❖ Python provides three types of access modifiers private, public, and protected.
- 1) **Public Member:** Accessible anywhere from outside oclass.
  - 2) **Private Member:** Accessible within the class
  - 3) **Protected Member:** Accessible within the class and its sub-classes

# Access Modifiers in Python

Access modifiers are used to limit access to the variables and functions of a class.

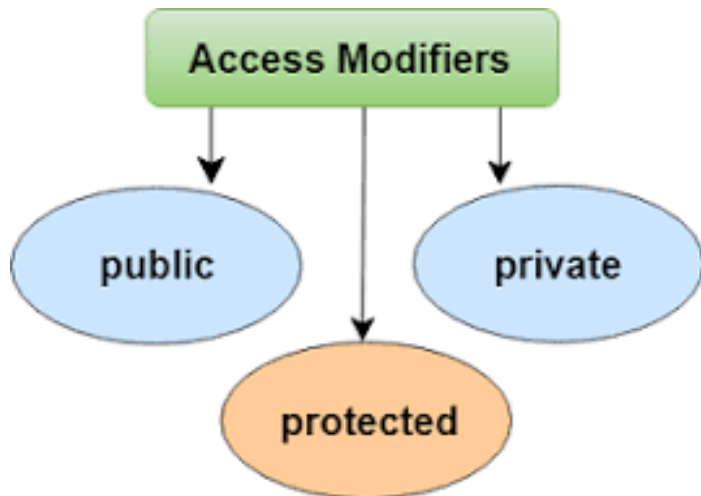
- Public -> By Default
- Protected -> `_method/variable`
- Private -> `__method/varoable`



**Ini juga menyangkut penggunaan folder/directory di dalam manage Python files.**

# Access Modifiers

| Access Modifiers | Same Class | Same Package | Sub Class | Other Packages |
|------------------|------------|--------------|-----------|----------------|
| <i>Public</i>    | Y          | Y            | Y         | Y              |
| <i>Protected</i> | Y          | Y            | Y         | N              |
| <i>Private</i>   | Y          | N            | N         | N              |



In most of the object-oriented languages access modifiers are used to limit the access to the variables and functions of a class and a package/folder.

# Data hiding using access modifiers

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

```
        self._project = project
```

```
        self.__salary = salary
```

Public Member (accessible within or outside of a class)

Protected Member (accessible within the class and it's sub-classes)

Private Member (accessible only within a class)

Data Hiding using Encapsulation

Ingat ini sangat penting, karena dapat **mencegah perubahan** yang tidak disengaja. Bukan tidak dapat di ubah, namun, variabel objek tersebut hanya dapat di ubah dengan metode objek.

# Data hiding using access modifiers

```
class Employee:
```

```
    def __init__(self, name, salary):
```

```
        self.name = name
```

Public Member (accessible within or outside of a class)

```
        self._project = project
```

Protected Member (accessible within the class and it's sub-classes)

```
        self.__salary = salary
```

Private Member (accessible only within a class)

Data Hiding using Encapsulation

Dapat diperhatikan pada contoh di atas : variable *name*, *\_project* dan *\_\_salary*.

- 1) **Without underscore:** Public variable, dapat di akses dari dalam maupun luar class.
- 2) **Single underscore:** Protected variable, tidak seharusnya di akses secara langsung. Namun, tidak menghentikan Python untuk mengeksekusi perintah.
- 3) **Double underscore:** Private variable, sulit untuk di akses, namun masih memungkinkan.

# Private Methods *in* Class Python

```
1  # Parent Class
2  class Human:
3      # Declaring public method
4      def funny(self):
5          print("Public method")
6
7      # Declaring private method
8      def __funny(self):
9          print("Private method")
10
11 # Child Class
12 class Teacher(Human):
13     # Method #1
14     def call_public(self):
15         # Calling public method of parent class
16         print("\nInside Teacher class")
17         self.funny()
18
19     # Method #2
20     def call_private(self):
21         # Calling private method of parent class
22         self.__funny()
```

Private method adalah method yang tidak boleh diakses di luar class.

To define a private method prefix the member name with the double underscore “\_\_”.

```
25 # Parent Class
26 obj1 = Human()
27 obj1.funny()      # Calling public method
28 obj1.__funny()    # raise an AttributeError
29
30 # Child Class
31 obj2 = Teacher()
32 obj2.call_public() # Calling public method
33 obj2.call_private() # raise an AttributeError
```

Menurut kalian, jika private method tidak bisa di akses dari luar class, lalu bagaimana cara untuk bisa mengakses private methods dari luar class menggunakan object?



**Apakah private method bisa diwariskan dalam OOP python?**  
Metode private tidak dapat diakses di kelas turunan karena detail implementasi internal harus disembunyikan dari kelas turunan untuk menjaga fleksibilitas dan kestabilan kode.

# How to Access Private Methods?

```
# Creating a class
2 class Student:
3
4     # Declaring public method
5     def funny(self):
6         print("Public method")
7
8     # Declaring private method
9     def __funny(self):
10        print("Private method")
11
12    # Declaring public method
13    def Help(self):
14        self.funny()    # public method
15        self.__funny() # private method
16
17    # Create object
18    obj_semmy = Student()
19    obj_semmy.Help() # public method
```

Private methods can be accessed by calling the private methods via public methods.

# Benefits of Encapsulation

|                                                      |                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Encapsulation can be applied for Security            | The main advantage of using encapsulation is the <b>security of the data</b> . <b>Encapsulation protects an object from unauthorized access</b> . It allows <b>private and protected access levels to prevent accidental data modification</b> .                                                     |
| Encapsulation can be applied for Data Hiding         | The <b>user would not be knowing what is going on behind the scene</b> . They would only be knowing that <b>to modify a data member, call the setter method</b> . To <b>read a data member, call the getter method</b> . <b>What these setter and getter methods are doing is hidden from them</b> . |
| Encapsulation can be applied for Simplicity          | It simplifies the <b>maintenance of the application</b> by keeping classes separated and preventing them from tightly coupling with each other.                                                                                                                                                      |
| Encapsulation can be applied for Aesthetics/estetika | Bundling data and methods within a class <b>makes code more readable and maintainable</b> .                                                                                                                                                                                                          |

# Benefit Access Modifiers (Encapsulation)

**Apa manfaat *encapsulation* (access modifiers) dalam pengembangan sistem berbasis web development?**

The screenshot displays the 'Active Classes' page of a web application. The interface includes a sidebar with navigation links (Home, Informations, Search All Classes, Search Students, Sign Out) and a main content area. The main area features a table of active classes with columns for Course, Meetings/Room, Schedule, Status, and Actions. A notification banner at the top of the table area states: 'Untuk kesesuaian RPS, diharapkan mengisi topik pembahasan di class pada setiap jadwal presensi.'

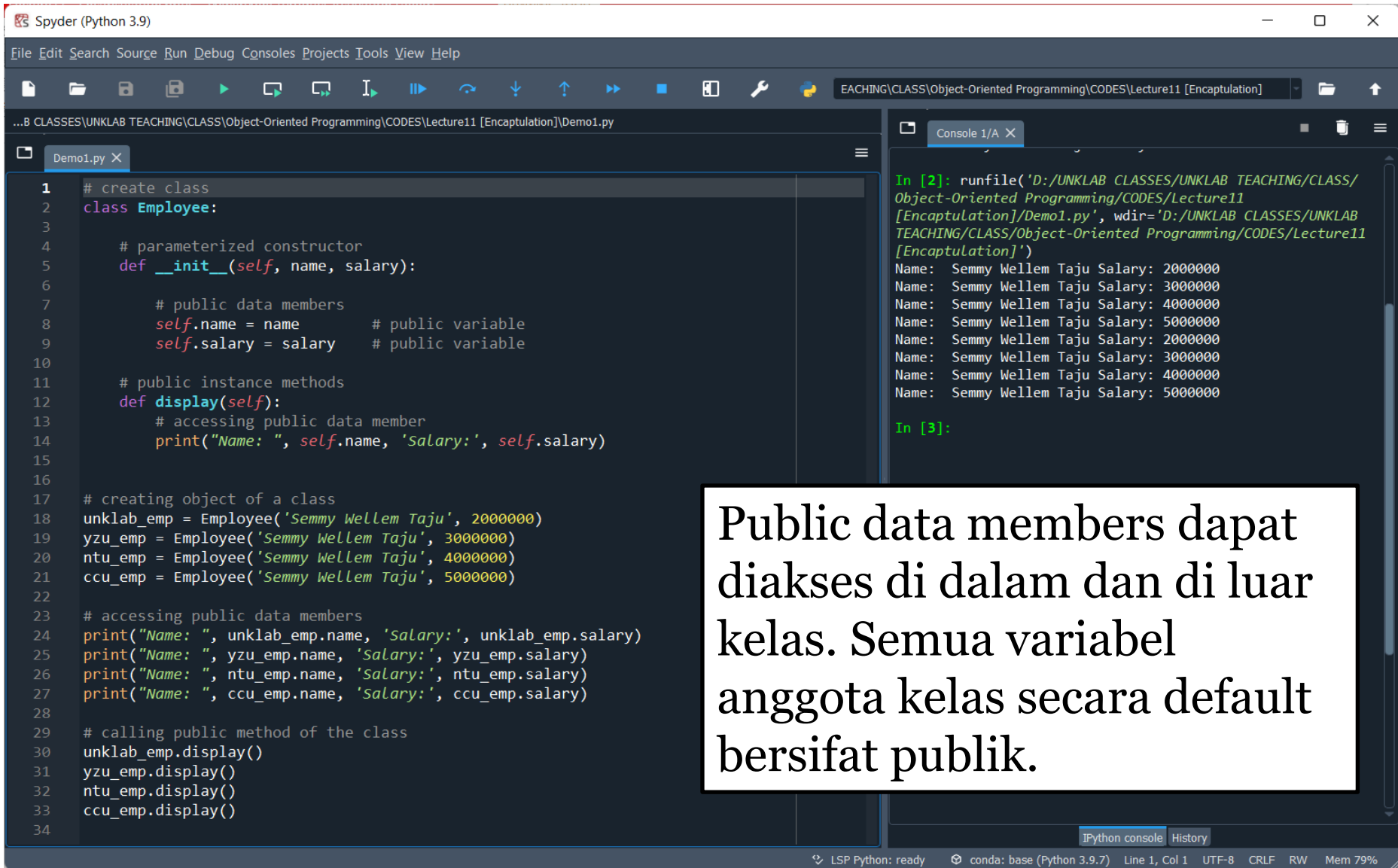
| Course                                                                                                                                                                   | Meetings/Room                               | Schedule                      | Status | Actions |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-------------------------------|--------|---------|
| <b>[IFKV332] Artificial Intelligence - B</b><br>31 Students<br>Semester Genap 2024/2025<br>3 SKS (7 absences max limit)<br>Fakultas Ilmu Komputer<br>Taju, Semmy Wellem  | 7 Public Scheduled<br>GK2 Computer Lab 1    | Senin-Rabu<br>10.10 - 11.30   | active | Options |
| <b>[IFMI352] Back-End Web Development - D</b><br>37 Students<br>Semester Genap 2024/2025<br>3 SKS (7 absences max limit)<br>Fakultas Ilmu Komputer<br>Taju, Semmy Wellem | 8 Public Scheduled<br>GA - 203 Computer Lab | Senin-Rabu<br>14.40 - 16.00   | active | Options |
| <b>[IFRL511] DevOps Engineering - A</b><br>27 Students<br>Semester Genap 2024/2025<br>3 SKS (7 absences max limit)<br>Fakultas Ilmu Komputer<br>Taju, Semmy Wellem       | 9 Public Scheduled<br>GA - 203 Computer Lab | Selasa-Kamis<br>14.40 - 16.00 | active | Options |



# Exercise *for* Students

Implement Encapsulation with a Class in Python

# Public Access Modifier



```
1 # create class
2 class Employee:
3
4     # parameterized constructor
5     def __init__(self, name, salary):
6
7         # public data members
8         self.name = name        # public variable
9         self.salary = salary    # public variable
10
11     # public instance methods
12     def display(self):
13         # accessing public data member
14         print("Name: ", self.name, 'Salary:', self.salary)
15
16
17 # creating object of a class
18 unklab_emp = Employee('Semmy Wellem Taju', 2000000)
19 yzu_emp = Employee('Semmy Wellem Taju', 3000000)
20 ntu_emp = Employee('Semmy Wellem Taju', 4000000)
21 ccu_emp = Employee('Semmy Wellem Taju', 5000000)
22
23 # accessing public data members
24 print("Name: ", unklab_emp.name, 'Salary:', unklab_emp.salary)
25 print("Name: ", yzu_emp.name, 'Salary:', yzu_emp.salary)
26 print("Name: ", ntu_emp.name, 'Salary:', ntu_emp.salary)
27 print("Name: ", ccu_emp.name, 'Salary:', ccu_emp.salary)
28
29 # calling public method of the class
30 unklab_emp.display()
31 yzu_emp.display()
32 ntu_emp.display()
33 ccu_emp.display()
34
```

Console 1/A X

```
In [2]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture11
[Encaptulation]/Demo1.py', wdir='D:/UNKLAB CLASSES/UNKLAB
TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture11
[Encaptulation]')
Name: Semmy Wellem Taju Salary: 2000000
Name: Semmy Wellem Taju Salary: 3000000
Name: Semmy Wellem Taju Salary: 4000000
Name: Semmy Wellem Taju Salary: 5000000
Name: Semmy Wellem Taju Salary: 2000000
Name: Semmy Wellem Taju Salary: 3000000
Name: Semmy Wellem Taju Salary: 4000000
Name: Semmy Wellem Taju Salary: 5000000

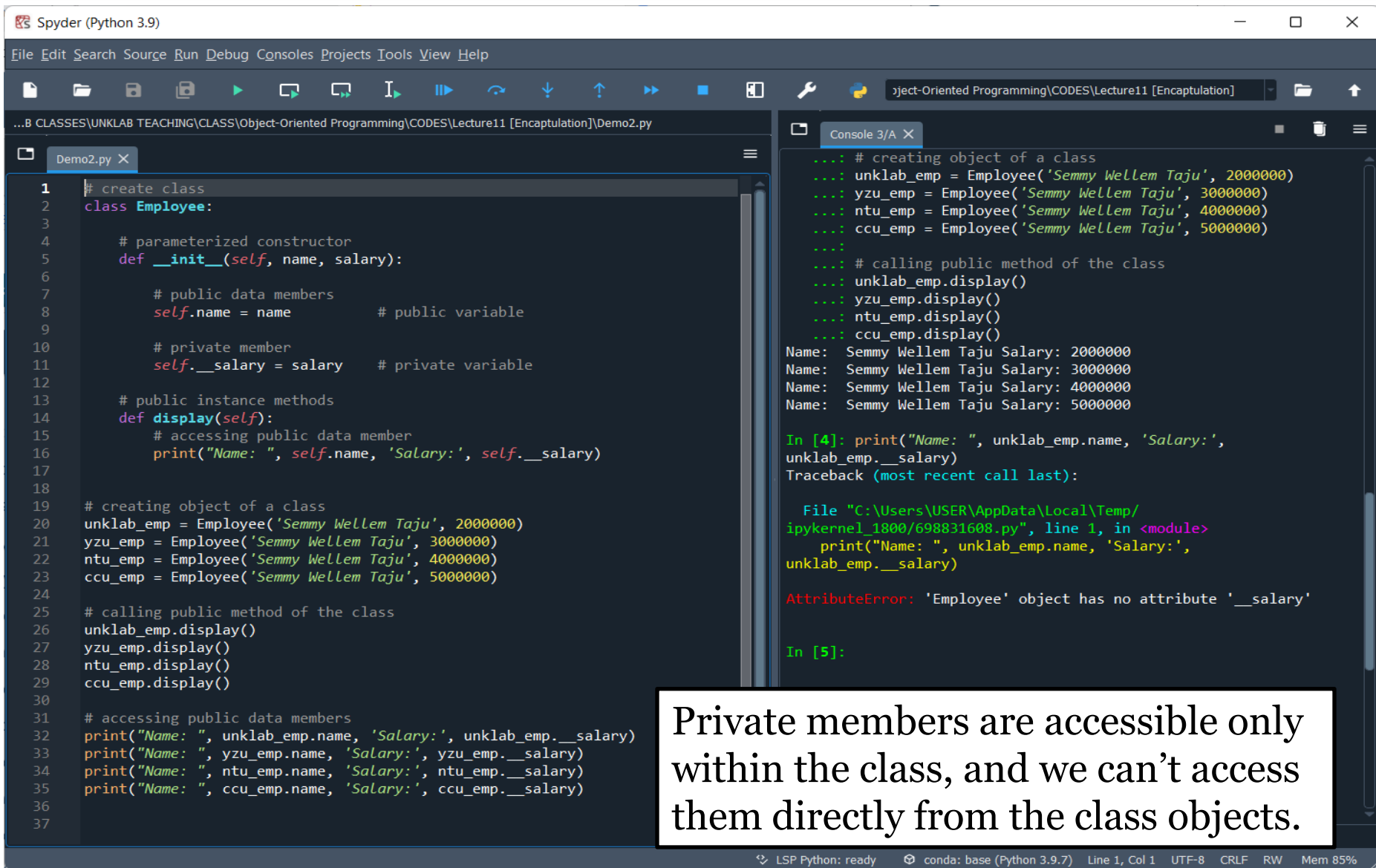
In [3]:
```

IPython console History

LSP Python: ready conda: base (Python 3.9.7) Line 1, Col 1 UTF-8 CRLF RW Mem 79%

Public data members dapat diakses di dalam dan di luar kelas. Semua variabel anggota kelas secara default bersifat publik.

# Private Access Modifier



The screenshot shows the Spyder Python IDE interface. The left pane displays a Python script named `Demo2.py` with the following code:

```
1 # create class
2 class Employee:
3
4     # parameterized constructor
5     def __init__(self, name, salary):
6
7         # public data members
8         self.name = name           # public variable
9
10        # private member
11        self.__salary = salary     # private variable
12
13    # public instance methods
14    def display(self):
15        # accessing public data member
16        print("Name: ", self.name, 'Salary:', self.__salary)
17
18
19    # creating object of a class
20    unklab_emp = Employee('Semmy Wellem Taju', 2000000)
21    yzu_emp = Employee('Semmy Wellem Taju', 3000000)
22    ntu_emp = Employee('Semmy Wellem Taju', 4000000)
23    ccu_emp = Employee('Semmy Wellem Taju', 5000000)
24
25    # calling public method of the class
26    unklab_emp.display()
27    yzu_emp.display()
28    ntu_emp.display()
29    ccu_emp.display()
30
31    # accessing public data members
32    print("Name: ", unklab_emp.name, 'Salary:', unklab_emp.__salary)
33    print("Name: ", yzu_emp.name, 'Salary:', yzu_emp.__salary)
34    print("Name: ", ntu_emp.name, 'Salary:', ntu_emp.__salary)
35    print("Name: ", ccu_emp.name, 'Salary:', ccu_emp.__salary)
36
37
```

The right pane shows the console output for the script. It displays the output of the `display()` method for each object, followed by an attempt to access the private attribute `__salary` directly from the `unklab_emp` object, which results in an `AttributeError`:

```
... # creating object of a class
... unklab_emp = Employee('Semmy Wellem Taju', 2000000)
... yzu_emp = Employee('Semmy Wellem Taju', 3000000)
... ntu_emp = Employee('Semmy Wellem Taju', 4000000)
... ccu_emp = Employee('Semmy Wellem Taju', 5000000)
...
... # calling public method of the class
... unklab_emp.display()
... yzu_emp.display()
... ntu_emp.display()
... ccu_emp.display()
Name: Semmy Wellem Taju Salary: 2000000
Name: Semmy Wellem Taju Salary: 3000000
Name: Semmy Wellem Taju Salary: 4000000
Name: Semmy Wellem Taju Salary: 5000000

In [4]: print("Name: ", unklab_emp.name, 'Salary:',
unklab_emp.__salary)
Traceback (most recent call last):

  File "C:\Users\USER\AppData\Local\Temp\ipykernel_1800\698831608.py", line 1, in <module>
    print("Name: ", unklab_emp.name, 'Salary:',
unklab_emp.__salary)

AttributeError: 'Employee' object has no attribute '__salary'

In [5]:
```

A text box at the bottom right of the image contains the following text:

Private members are accessible only within the class, and we can't access them directly from the class objects.

Tambahkan **2 private method**  
( *\_\_showroom()* dan *\_\_showstatus()* )  
di dalam class Employee, dua private method  
tersebut hanya menampilkan text biasa.  
Selanjutnya students diminta untuk temukan  
cara untuk bisa mengakses dua private  
method tersebut dari luar class  
menggunakan object *unklab\_emp*, *yzu\_emp*,  
*ntu\_emp* dan *ccu\_emp*?

# END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

